

JARVIS Codebase Audit

Audit baseline: ARCHITECTURE.md v3.1 (14 May 2026) | Audit date: 16 May 2026 | Auditor: Claude Opus 4.7 | Repo: jarvis @ main (afb35f2)

Executive Summary

Phase 1A is technically running — typed input reaches OpenAI and a reply renders. Against the v3.1 architecture, however, the current code is a thin chat scaffold, not Phase 1A of JARVIS. The non-negotiable Phase 0 prerequisites (cost guard, telemetry, prompt versioning, provider abstraction) are missing, the runtime stack (Next.js/TypeScript) diverges from the architecture's stated stack (Python/FastAPI), and a critical secret-handling issue requires immediate action.

One action this hour: Rotate both API keys. They were transmitted to this audit in plain text. Even if they were never git-committed, they have left the local boundary.

Verdict: Do *not* add the Anthropic provider on top of the current shape. Land the Phase 0 prerequisites (cost guard, telemetry, provider interface, streaming, request validation) first — what this audit calls Phase 1.5. Adding Anthropic to today's code locks in the abstraction debt and makes Phase 1C (registry) and Phase 1D (router) significantly harder.

Finding #1 — CRITICAL: Exposed API Keys

What: The contents of `.env.local` — including both the OpenAI `sk-proj-...` key and the Anthropic `sk-ant-...` key — were included verbatim in the audit context I was given. `.gitignore` covers `.env*`, so likely not committed, but the keys have nonetheless left their original boundary.

Immediate actions (do these before continuing the audit response):

- Revoke the OpenAI key at `platform.openai.com/api-keys` and issue a new one.
- Revoke the Anthropic key at `console.anthropic.com/settings/keys` and issue a new one.
- Review OpenAI and Anthropic usage logs for any calls you don't recognise.
- Verify `git log --all -p -- .env.local` shows no history (it should not, given `.gitignore`, but verify).
- Going forward, never paste `.env*` contents into prompts. Provide the file *path* and let the tool read it.

Also note: `process.env.OPENAI_API_KEY || ""` in `src/lib/config.ts` silently accepts an empty key. Replace with a fail-fast assertion at module load so a missing key crashes the boot instead of producing confusing 401s at runtime.

1. Alignment Matrix — v3.1 vs Repo

Architecture requirement (v3.1)	Repo state	Status
Cost cap module (Phase 0, P0)	Absent. No daily/weekly/monthly caps. No budgeting on /api/chat.	MISSING
Telemetry / observability	console.error only. No SQLite events table, no console.log latency log.	MISSING
Prompt versioning under prompts/	prompts/.gitkeep only. No system prompt loaded in system call.	MISSING
Provider abstraction	Single function generateOpenAIResponse called in BufferContent.	INSUFFICIENT
Model registry (YAML, source of truth)	Hardcoded in src/lib/config.ts. No tier metadata, no capabilities, no swap path.	MISSING
Router (intent / safety / capability / cost)	None. POST handler does not classify, does not type, does not select.	MISSING
Streaming by default	openai.ts uses non-streaming chat.completions.create. Route returns one JSON blob.	MISSING
Safety gates / confirmation flow	None. No destructive actions yet, but no scaffolding in place (acceptable now, must precede)	MISSING
State is explicit / bounded persistence	Chat history in client useState only. Lost on refresh. Bounded growth in payload.	BROKEN
Repo structure (apps/, core/, models/, tools/, etc.)	Next.js app structure. No core/, no models/providers/, no tools/. (Intentional? unclear)	DIRTY/ROTTEN
Stack: Python + FastAPI + Ollama	Next.js 16 + React 19 + TypeScript + openai SDK	CONTRADICTION (README still claims Python)
Pre-commit hooks + test framework (Phase 1, P1)	No tests, no hooks, no husky, no vitest, no playwright	MISSING
AGENTS.md: 'NOT the Next.js you know'	No evidence node_modules/next/dist/docs/ was converted. Route uses generic patterns.	UNVERIFIED

2. Security Issues

2.1 Secret handling (see Finding #1)

- Keys exposed via paste — rotate now.
- `config.ts` uses `process.env.OPENAI_API_KEY || ""`. Fail fast instead: throw at load if key missing.
- README setup instructions reference `.env`; the repo actually uses `.env.local`. Both are gitignored, but the mismatched convention is the kind of thing that produces an accidental commit when a future contributor names theirs `.env.production`.

2.2 Unbounded request & cost-drain surface

- **No rate limiting on /api/chat.** Anyone who reaches localhost (or your eventual deployed URL) can hit the route in a tight loop. Combined with no cost cap, this is unbounded spend exposure.
- **No request size limit.** `body.messages` is accepted as-is. A 200KB payload of crafted messages becomes input tokens on every send.
- **No origin / CORS check.** Next.js route handlers accept POSTs from any origin by default. Browser CORS protects from cross-site fetch but does not protect from a hostile page issuing `fetch(...)` as no-cors or from non-browser clients.
- **No authentication.** Local dev is fine, but the architecture (§2.5) requires safety gates as core, not future. Without an auth layer, the safety classifier has nothing to gate against. Add a session token now — even a single-user shared secret — before deploying anywhere.

2.3 Input validation & injection

- **No runtime validation.** `const body: ChatRequest = await req.json()` is a TypeScript cast, not a check. Malformed input reaches the OpenAI SDK and produces opaque 500s. Use Zod or Valibot.
- **No prompt-injection defence.** User content is appended verbatim to the message array. There is no system message instructing the model to ignore embedded instructions, and no allow-list of tool calls. Acceptable today (no tools exist), but the day you wire Phase 2 desktop tools, an unguarded prompt injection becomes a code-execution path.

2.4 Error disclosure

- Route catches all errors and returns the literal string "Something went wrong." That's good (no stack leak), but (a) the client renders that string as a normal assistant message, which destroys trust in the output, and (b) you lose the ability to distinguish auth failures, validation failures, rate-limit hits, and upstream 5xxs at the UI layer.

3. Architecture Weaknesses vs v3.1

3.1 Phase 0 prerequisites unmet

v3.1 §16 is explicit: *"Cost caps implemented before Phase 1A closes. The orchestrator refuses requests that would breach caps. Non-negotiable."* Phase 1A is currently running cloud calls with no cap, no telemetry, no router. By the architecture's own standard, Phase 1A is not actually closed.

3.2 Stack vs architecture contradiction

The architecture (§10 Repository Structure, §20 Hardware Reality Check) and the README both describe a Python runtime — `jarvis.py`, FastAPI under `apps/api/`, Ollama in-process, SQLite, pytest. The actual repo is a Next.js 16 web app with the OpenAI Node SDK. These are not the same system. Two paths forward:

- **Path A — Web is the runtime, architecture follows.** Rewrite the architecture's repo layout and stack table to reflect a TypeScript-first stack: `core/` becomes `src/core/`, Ollama is called over HTTP from Next, telemetry uses `better-sqlite3` or `libsql`, voice runs via WebSpeech / Web Audio. Lose the Electron desktop assumption; keep the principles.
- **Path B — Python is the runtime, web is a UI.** Keep the Python core as planned; this Next app becomes `apps/dashboard/` talking to a local FastAPI server. The current chat endpoint moves to a Python service; the Next route becomes a thin proxy.

Either is defensible. **Neither is the current state.** Decide before Phase 1B so the provider abstraction is built in the right language.

3.3 No system prompt

Messages are sent to OpenAI as-is. There is no JARVIS persona, no safety preamble, no tool-use frame, no token ceiling. Architecture §18.1 requires prompts to live under `prompts/` with hash-logged loading per call. Today, `prompts/jarvis_system.md` does not exist.

3.4 No router

Every request — "hi", "turn off the light", "review this 4kloc repo" — routes to the same `gpt-4o-mini` call. Architecture §4 specifies a four-stage router. None of it exists. Adding it after a second provider is significantly harder than adding it before.

3.5 Unbounded conversation history

Every send re-uploads the entire messages array. By turn 50 your input tokens dominate your bill. v3.1 §15 mandates a memory compression layer; v3.1 §8.1 expects sentence-streamed responses. Neither exists. This is the single biggest cost-runaway risk in the current code.

3.6 Wrong Anthropic model id

`config.ts` sets `anthropic.model = "claude-opus-4-1"`. The current latest is `claude-opus-4-7` (per the runtime model info this session). Architecture §5 example uses `claude-opus-4-6`. Trivial fix, but a symptom of the deeper issue: model identifiers should live in a registry, not hardcoded in `config.ts`.

4. TypeScript & Next.js Mistakes

4.1 tsconfig path mapping

Current: `"@/*": ["./*"]`. This makes the import `@/src/lib/openai` rather than the conventional `@/lib/openai`. Either map `@/*` to `./src/*`, or introduce a second alias `~/*`. Decide before the import graph grows.

4.2 Leftover scaffolding

- `app/layout.tsx: metadata.title = "Create Next App"`. Replace with "JARVIS".
- `app/layout.tsx` loads Geist via `next/font/google` and sets CSS variables, but `app/globals.css` uses `font-family: Arial, Helvetica, sans-serif`. Geist is downloaded but never applied.
- `globals.css` defines light/dark CSS variables, but `app/page.tsx` hardcodes `bg-black text-white`. Either drop the variables or use them.
- `app/favicon.ico` is still the Next.js default.

4.3 API route

- Type cast without validation: `const body: ChatRequest = await req.json()`. Use Zod.
- No explicit runtime/dynamic directives. Recommended: `export const runtime = 'nodejs'; export const dynamic = 'force-dynamic';` — the OpenAI SDK requires Node runtime and explicit dynamic prevents Next from attempting any caching of the POST.
- Generic catch-all error: distinguish auth (401), validation (400), upstream timeout (504), rate-limit (429).

4.4 Client page (`app/page.tsx`)

- No try/catch around `fetch`. A network drop produces an unhandled rejection.
- On 500, route returns `{message: 'Something went wrong.'}`; client renders that as an assistant message. Check `res.ok` first, render an error state instead.
- Input and Send button are not disabled while `loading=true`. Spam-send is possible.
- No `AbortController`; user cannot cancel a slow request.
- `React key={index}`. Acceptable today; will break the moment you add edit/regenerate. Assign stable IDs at message creation.
- No auto-scroll to bottom on new message.
- Enter sends but Shift+Enter does not newline (single-line `<input>` instead of `textarea`).
- No empty-state or loading skeleton; the page can flash before `useState` initialises.

4.5 Tooling gaps

- `target: "ES2017"` in `tsconfig` is from an old template. Next 16 + React 19 + Node 20 can safely use ES2022.
- No `engines` field in `package.json`; Node version isn't pinned.
- No Prettier, no Husky, no lint-staged, no pre-commit hook. v3.1 Phase 1 P1 explicitly requires this.

- "lint": "eslint" with no path. "eslint ." is clearer and matches CI expectations.
- No test runner (vitest), no Playwright skeleton.

5. Scalability Problems

- **Conversation re-upload.** Every send re-transmits the full history. Linear cost growth per turn, quadratic across a session. Needs (a) Anthropic prompt-caching when that provider lands, and (b) a memory compression layer (v3.1 §15) that summarises old turns.
- **No persistence.** Chat history lives in React state. Refresh = data loss. SQLite session table from §17 telemetry schema solves this.
- **No streaming.** The client waits for the full response before rendering. At T3 latency targets (§7, ~1.5s P95) this is tolerable; at T4 frontier (3s P95, 6s ceiling) the UI will look frozen.
- **No concurrency control.** If you open two tabs and send simultaneously, both hit the route, both bill, neither knows about the other. No queue, no dedupe, no idempotency key.
- **No caching.** Identical questions cost the same as fresh ones. v3.1 §16 calls for caching repeated results.

6. Is the Provider Abstraction Ready for Anthropic?

Short answer: No. Adding Anthropic on top of the current shape would calcify the wrong abstraction.

Current shape:

```
// src/lib/openai.ts
export async function generateOpenAIResponse(messages: Message[]) {
  const response = await openai.chat.completions.create({
    model: config.openai.model,
    messages,
  });
  return response.choices[0]?.message?.content ?? 'No response generated.';
}

// app/api/chat/route.ts
const response = await generateOpenAIResponse(body.messages);
```

The route knows the provider's name. The function name encodes it. Adding `generateAnthropicResponse` means the route grows a switch statement. That switch grows again at Ollama. By the time Phase 1C registry lands, the route has three branches and a config flag.

What 'good enough' looks like before Anthropic lands:

```
// src/lib/providers/types.ts
export interface GenerateOptions {
  model: string;
  systemPrompt?: string;
  maxTokens?: number;
  temperature?: number;
}
export interface GenerateResult {
  content: string;
  inputTokens: number;
  outputTokens: number;
  costUsd: number;
  latencyMs: number;
}
export interface ChatProvider {
  readonly id: string; // 'openai' | 'anthropic' | 'ollama'
  generate(messages: Message[], opts: GenerateOptions): Promise<GenerateResult>;
```

```
    stream(messages: Message[], opts: GenerateOptions): AsyncIterable<string>;  
  }  
  
  // src/lib/providers/openai.ts      implements ChatProvider  
  // src/lib/providers/anthropic.ts   implements ChatProvider  
  // src/lib/providers/index.ts       registry: id -> ChatProvider
```

The route then becomes: `const provider = registry.get(modelId); const result = await provider.generate(messages, opts);` Cost is reported by the provider, not the route. Streaming is uniform. Telemetry has one place to fire.

7. Fix List Before Phase 1.5

Phase 1.5 = "the bridge from a chat scaffold to something that earns the name JARVIS". It is not in v3.1 explicitly; I'm proposing it as a hard gate before Phase 1B (provider wrappers) starts.

P0 — Blockers (this week)

#	Fix	Why
1	Rotate exposed API keys	Finding #1
2	Implement cost guard module (hard daily/weekly/monthly caps, orches...	v3.1 §16, v3.1 §24
3	Zod validation on /api/chat request body	Security 2.3
4	Rate limit /api/chat (in-memory bucket is fine for now)	Security 2.2
5	Fix client error path: read res.ok, render error state, do not render error...	TS/NDJSON as assistant message
6	Throw at boot if API keys missing (no silent empty string)	Security 2.1
7	Add prompts/jarvis_system.md and load on every call; log prompt hash	v3.1 §18.1

P1 — Architectural prerequisites for Phase 1B

#	Fix	Why
8	Decide stack: Path A (TS-first) or Path B (Python core + Next UI). Update AR...	Section 3.2
9	Define ChatProvider interface in src/lib/providers/types.ts	Section 6
10	Refactor openai.ts to implement ChatProvider	Section 6
11	Add streaming: ReadableStream from route, SSE or NDJSON to client	v3.1 §8
12	Add SQLite telemetry (better-sqlite3 or libsql) with the v3.1 §17 event...	v3.1 §17
13	Add registry.ts (or .yaml + loader). Replace hardcoded model strings...	v3.1 §5
14	Fix Anthropic model id (→ claude-opus-4-7 or the registry-current id)	Section 3.6
15	Cap conversation length OR add first-pass summarisation at N=20	Section 5 / v3.1 §15
16	Persist sessions to SQLite, not React useState	v3.1 §9
17	Fix tsconfig path mapping (@/* → ./src/*)	TS/Next 4.1

P2 — Polish before 1B is merged

#	Fix	Why
18	Husky + lint-staged + Prettier pre-commit hook	v3.1 Phase 1 P1
19	Vitest skeleton + 3 unit tests (provider stub, validation, cost calc)	v3.1 §18
20	Playwright smoke test: send a message, assert reply renders	v3.1 §18
21	Reconcile README: drop Python-only narrative or move it to docs/	Section 2.1
22	Replace 'Create Next App' metadata, Geist font wiring, color inconsis...	TS/Next 4.2

#	Fix	Why
23	Add engines.node to package.json; upgrade tsconfig target to ES2022	Next 4.5
24	Wire AGENTS.md rule: confirm each Next.js change was checked against	AGENT_Side_modules/next/dist/docs/

8. Does the v3.1 Roadmap Order Still Make Sense?

Mostly yes, with two friction points that the current code surfaces.

8.1 Phase 0 is not actually done

The roadmap treats Phase 0 as in-progress and Phase 1A as complete. The audit finds the opposite: 1A's user-visible behaviour works, but 1A's *structural* exit criteria (cost caps enforce themselves; tests run; pre-commit hook exists) are missing. The bar for 'phase complete' should include the structural criteria, not just the demo. Recommendation: re-open Phase 0 + 1A; close them together with the P0/P1 fix list above.

8.2 Phase 1B before any more UI features

Every feature added to the current chat UI before the provider interface lands becomes a thing you have to re-test against three providers. Freeze UI work above the API line until the abstraction is in.

8.3 Phase 2 has a stack problem

Phase 2 is "local desktop tools" (open folders, launch apps, file ops). A Next.js web app cannot do this from the browser. v3.1 §10 implies an Electron desktop shell (apps/desktop/); the current repo has no such shell. Phase 2 will require either: (a) an Electron wrapper, (b) a local FastAPI/Node sidecar the web app calls, or (c) a CLI runtime that the web UI proxies to. **Pick this before Phase 2 starts**, ideally as part of the stack decision in P1 item #8.

8.4 Phase 4 voice depends on Phase 2 tools

Voice without a tool surface is a transcription toy. The roadmap already orders 2 before 4 — correct. Just confirming this audit doesn't suggest swapping them.

8.5 Suggested revised early order

Phase	Capability	Status after this audit
0	Foundations + cost caps + tests + pre-commit + Docker	RE-OPEN. P0/P1 items 1–7.
1A	Typed core loop	RE-OPEN. Close jointly with 0.
1.5	Provider interface, streaming, telemetry, registry skeleton	NEW. P1 items 8–17.
1B	Anthropic + Ollama providers behind the interface	Unblocked by 1.5.
1C	Registry calibration suite	As planned.
1D	Router (intent / safety / capability / cost)	As planned.
Stack-decision gate	Resolve web-vs-desktop before Phase 2	NEW. Output of P1 item 8.
2	Local tools (via the chosen runtime)	As planned, contingent on gate.
3–9	As v3.1	Order unchanged.

9. Closing Notes

The code that exists is clean and readable for what it is: a 200-line Next chat. The gap between it and v3.1 is not about code quality; it's about scope. v3.1 describes an orchestration system with a chat UI on top. The repo

today is a chat UI with no orchestration underneath. The fix list above is the smallest set of changes that turns one into the other without cutting any planned capability.

Three things to do today: (1) rotate the keys, (2) decide Path A or Path B for the stack, (3) start the cost guard module. Everything else in the P0/P1 list follows from those three.

— Audit complete.